

INDIAN INSTITUTE OF TECHNOLOGY HYDERABAD

M.Tech Computer Science and Engineering

**CS6890 FRAUD ANALYTICS USING PREDICTIVE AND SOCIAL NETWORK
TECHNIQUES**

Graph Neural Network Node Classification

Assignment 1

Akarsh Dubey CS25MTECH14001

Atish Kadam CS25MTECH14003

April 5, 2026

Code:

[https:](https://colab.research.google.com/drive/1nEXSDcI5BI8570YbFa_EImJE6mZK0ku?usp=sharing)

[//colab.research.google.com/drive/1nEXSDcI5BI8570YbFa_EImJE6mZK0ku?usp=sharing](https://colab.research.google.com/drive/1nEXSDcI5BI8570YbFa_EImJE6mZK0ku?usp=sharing)

Contents

1	Introduction	3
2	Dataset Description	3
3	Methodology	3
3.1	Graph Convolutional Network (GCN) — From Scratch	3
3.2	Graph Attention Network (GAT) — From Scratch	4
3.3	GraphSAGE — Library Implementation	5
4	Experimental Setup	5
4.1	Hyperparameters	5
4.2	Training Procedure	6
4.3	Reproducibility	6
5	Results and Discussion	6
5.1	Training Progress	6
5.2	Test Set Results	6
5.3	Output Screenshot	6
5.4	Discussion	6
6	Conclusion	7

1. Introduction

Detecting fraudulent activity in large networks is a natural fit for Graph Neural Networks (GNNs), since fraud patterns often manifest through relationships rather than isolated node attributes alone. GNNs aggregate feature information across graph neighbourhoods, allowing each node to incorporate context from its surroundings when forming a classification decision.

This report covers the design, implementation, and evaluation of three GNN architectures applied to a binary node-classification task on a fraud-labelled graph:

- **GCN** (Graph Convolutional Network) — built from scratch using raw PyTorch scatter operations; no external GNN library used.
- **GAT** (Graph Attention Network) — built from scratch; multi-head attention coefficients computed via `einsum` and `scatter_reduce`.
- **GraphSAGE** — uses `SAGEConv` from PyTorch Geometric with a max aggregator, as permitted by the assignment specification.

Models are assessed on Accuracy, Precision, Recall, and F1-Score on a held-out test split.

2. Dataset Description

Three CSV files form the complete dataset:

File	Contents	Dimensions
<code>edges.csv</code>	Source–target node pairs	$8,000 \times 2$
<code>features.csv</code>	16 numeric attributes per node	$2,000 \times 17$
<code>labels.csv</code>	Binary fraud label (0 / 1) per node	$2,000 \times 2$

Both edge directions are added to make the graph undirected (16,000 edges total after symmetrisation). The class distribution is perfectly balanced (1,000 nodes per class).

Node features are standardised with `StandardScaler` fitted *only* on the training partition to prevent data leakage. The 60/20/20 stratified split produces:

Train	Validation	Test
1,200	400	400

Balanced class weights are passed to the loss function alongside label smoothing ($\epsilon = 0.05$) to encourage better-calibrated predictions.

3. Methodology

3.1 Graph Convolutional Network (GCN) — From Scratch

The GCN layer follows the symmetric normalisation scheme of Kipf & Welling (ICLR 2017). Given a feature matrix $H^{(l)}$ at layer l , the update rule is:

$$H^{(l+1)} = \sigma\left(\hat{D}^{-1/2} \hat{A} \hat{D}^{-1/2} H^{(l)} W^{(l)}\right)$$

where $\hat{A} = A + I$ incorporates self-loops and $\hat{D}$ is the corresponding degree matrix.

Key implementation choices:

1. Self-loop edges are concatenated to the edge index before degree computation.
2. Normalisation factors $\hat{d}_i^{-1/2} \hat{d}_j^{-1/2}$ are precomputed once per graph to avoid redundant work during each forward pass.
3. Message passing uses `scatter_add` on destination indices.
4. Each `GraphConvLayer` applies an ℓ_2 row-normalisation to its output for stability.
5. The full model uses **three** convolutional layers interleaved with batch normalisation and ReLU activations; dropout (0.5) is applied after each hidden layer.
6. During training, 10% of edges are randomly dropped per step (*edge-dropout augmentation*) to prevent over-reliance on specific graph paths.

3.2 Graph Attention Network (GAT) — From Scratch

GAT (Veličković et al., ICLR 2018) replaces fixed normalisation with learned, per-edge attention weights. For an edge (i, j) :

$$e_{ij} = \text{LeakyReLU}\left(\mathbf{a}^\top [Wh_i \parallel Wh_j]\right), \quad \alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}(j)} \exp(e_{kj})}$$

The aggregated embedding is $h'_j = \sigma\left(\sum_{i \in \mathcal{N}(j)} \alpha_{ij} Wh_i\right)$.

Key implementation choices:

1. Node projections use `torch.einsum("hio,ni->hno", W, x)` to handle multiple heads efficiently without looping.
2. Attention logits are decomposed into source and destination components ($\mathbf{a}_{\text{src}}, \mathbf{a}_{\text{dst}}$) for computational efficiency.
3. The softmax is computed in a numerically stable manner via per-head maxima gathered with `scatter_reduce (amax)` and subtracted before exponentiation.
4. Dropout is applied to the attention coefficients to prevent over-attending to specific edges.
5. The architecture has **three** attention layers; layers 1 and 2 use 4 heads with concatenation (hidden dim $16 \times 4 = 64$), while layer 3 uses a single head with mean pooling.

6. **Residual connections** project the input of each block and add it to the block output to stabilise deep training.

3.3 GraphSAGE — Library Implementation

GraphSAGE (Hamilton et al., NeurIPS 2017) inductively learns embeddings by sampling and aggregating neighbourhood features. The update for node v under the **max** aggregator is:

$$h'_v = W \cdot \left(h_v \parallel \max_{u \in \mathcal{N}(v)} h_u \right)$$

The max aggregator retains the most salient neighbourhood signal, which is particularly suited to fraud detection where even a single suspicious neighbour may be informative.

We use `SAGEConv(..., aggr="max")` from PyTorch Geometric across three layers (hidden dim 64). After layer 2, its output is added directly to layer 1’s output ($h \leftarrow h + h_{\text{res}}$) before being passed to the final classification layer, providing a shortcut gradient path.

4. Experimental Setup

4.1 Hyperparameters

Hyperparameter	Value
Learning rate	5×10^{-3}
Weight decay	5×10^{-4}
Dropout rate	0.5
Max epochs	400
Early stopping patience	40 epochs
LR scheduler	ReduceLROnPlateau (factor 0.5, patience 15)
Label smoothing	0.05
Edge dropout (train)	10%
Gradient clip norm	1.0

Model configurations:

Model	Architecture
GCN	3 layers, hidden size 64
GAT	3 layers, 4 heads in layers 1–2, 1 head in layer 3, hidden size 16 per head
GraphSAGE	3 layers, hidden size 64, max aggregator

4.2 Training Procedure

Each epoch performs a forward pass with edge-dropout-augmented edges, computes weighted cross-entropy loss with label smoothing, and updates parameters via Adam with gradient clipping. After each epoch, the full edge set is used for validation; if validation loss improves, model weights are checkpointed. The learning rate is halved automatically whenever validation loss stagnates for 15 epochs. Training terminates early if no improvement is seen for 40 consecutive epochs, and the best checkpoint is restored for test evaluation.

All metrics (Accuracy, Precision, Recall, F1) are computed with `average='weighted'` from `scikit-learn` on the 400-node test partition.

4.3 Reproducibility

- Global seeds set via `torch.manual_seed(42)` and `numpy.random.seed(42)`.
- Stratified splits use `random_state=42`.
- Environment: Google Colab, CPU runtime, Python 3.x, PyTorch 2.10.0.
- Full runnable code: [Colab Notebook](#).

5. Results and Discussion

5.1 Training Progress

Training logs at 50-epoch intervals (seed = 42):

Model	Epoch 50 (Train / Val / Gap)	Stop Epoch	Best Val Loss
GCN	0.2953 / 0.2705 / +0.0248	95	0.2703
GAT	0.2859 / 0.1441 / +0.1418	97	0.1376
GraphSAGE	0.1356 / 0.1289 / +0.0067	65	0.1205

5.2 Test Set Results

Model	Accuracy	Precision	Recall	F1-Score
GCN	0.9850	0.9852	0.9850	0.9850
GAT	0.9975	0.9975	0.9975	0.9975
GraphSAGE (PyG)	1.0000	1.0000	1.0000	1.0000

5.3 Output Screenshot

5.4 Discussion

All three models perform well above chance, confirming that graph-structural information adds meaningful signal for fraud detection beyond node features alone.

GCN scores lowest overall (≈ 0.985 across all metrics), stopping at epoch 95. Because spectral

Model	Acc	Precision	Recall	F1
GCN	0.9850	0.9852	0.9850	0.9850
GAT	0.9975	0.9975	0.9975	0.9975
GraphSAGE (PyG)	1.0000	1.0000	1.0000	1.0000

Figure 1: Model evaluation output from the Google Colab run.

normalisation divides each edge’s contribution equally by degree, high-degree hub nodes receive diluted signals, which may cause the model to miss localised fraud sub-structures.

GAT improves substantially (all metrics: 0.9975) by learning to down-weight uninformative neighbours and attend selectively to those carrying stronger fraud signal. It stops at epoch 97, slightly later than GCN, reflecting the larger parameter count of the multi-head attention mechanism; the substantially lower best validation loss (0.1376 vs. 0.2703) confirms the extra capacity is well-utilised.

GraphSAGE achieves perfect scores (1.000) and is the fastest to converge, stopping at epoch 65. The max-pooling aggregator is particularly well-suited here: even a single high-anomaly neighbour is sufficient to raise the representation of a potentially fraudulent node, and the aggregator preserves that extreme value rather than averaging it away. The residual connection between layers 1 and 2 ($h \leftarrow h + h_{\text{res}}$) provides an additional shortcut for gradient flow.

The consistent ranking **GraphSAGE** > **GAT** > **GCN** reflects a broader pattern: more expressive neighbourhood aggregation strategies that preserve or weight individual signals outperform smoothing-based methods on anomaly detection tasks.

6. Conclusion

Three GNN architectures were designed and evaluated for fraud-node classification on a 2,000-node graph. GCN and GAT were coded from scratch using PyTorch primitives; GraphSAGE leveraged PyTorch Geometric’s **SAGEConv**. Additional engineering — edge-dropout, label smoothing, gradient clipping, adaptive learning-rate scheduling, batch normalisation, and residual connections — was applied uniformly to improve training stability and generalisation.

GraphSAGE achieved perfect test performance (all metrics = 1.000), GAT reached 0.9975, and GCN reached ≈ 0.985 . The results highlight that neighbourhood aggregation strategy is the dominant factor for this task, with max-pooling proving most effective at surfacing anomalous signals in the graph. Future work could explore heterogeneous graph models or temporal extensions for streaming fraud detection.